

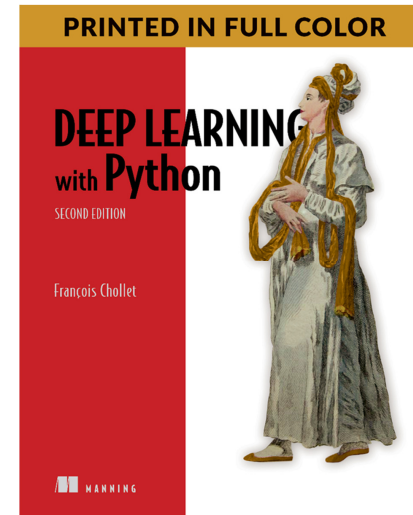
CS 59300

Application of Deep Learning Introduction to DL for Computer Vision

Instructor: Dr. Zesheng Chen

PURDUE
UNIVERSITY
FORT WAYNE

References



2

PURDUE
UNIVERSITY
FORT WAYNE

References

Machine Learning
(ML) 2021

李宏毅
Hung-yi Lee

- <https://speech.ee.ntu.edu.tw/~hylee/ml/2021-spring.php>
- https://www.youtube.com/watch?v=l4eLIsPM9Yc&ab_channel=Hung-yiLee

3

PURDUE
UNIVERSITY
FORT WAYNE

Computer Vision

- Is the problem domain that led to the **initial rise** of deep learning between 2011 and 2015.
- **Convolutional neural networks (CNN)** started getting remarkably good results on image classification competitions around that time.
 - **Convnets**
- DL models became **dominant** in the area of computer vision.

4



Applications of Deep Vision Models

- Applications
 - Autonomous driving
 - Robotics
 - AI-assisted medical diagnosis
 - Autonomous retail checkout systems
 - Autonomous farming
 - ...
- Vision / Image Data
 - Google Photos
 - Google image search
 - YouTube
 - Video filters in camera apps
 - OCR software
 -

5



Outline

- Introduction to CNN
 - “Hello World” MNIST Digits Example
- CNN for a Small Dataset
 - Classifying Images as Dogs or Cats Example

6



“Hello World” MNIST Digits Example

- MNIST dataset
 - Grayscale images of handwritten digits (28 x 28 pixels)
 - 60,000 training images and 10,000 test images
- Goal: classify images into their 10 categories (0 through 9)
- It has been shown that with a densely connected network, the test accuracy is **97.85%**
- Example using CNN

7

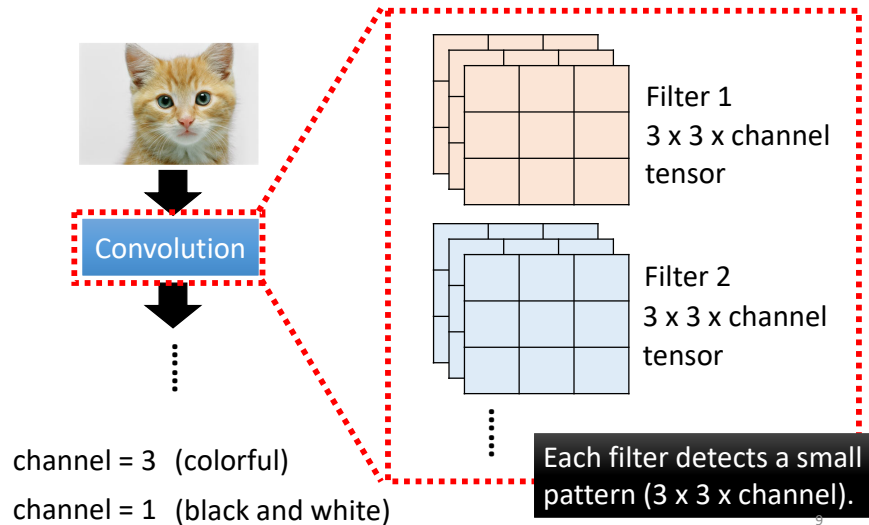


Conv2D Layer

- Both input and output are a rank-3 tensor
 - Shape (height, width, **channels**)
 - The number of channels for the output is determined by the “filters” argument
- Filters
 - A filter reflects a **feature** or a **pattern** in an image
 - The size of a filter is usually 3 x 3 x input_channels or 5 x 5 x input_channels
 - The number of filters decides the number of output channels
 - The numbers in the filters are **trainable weights**.

8

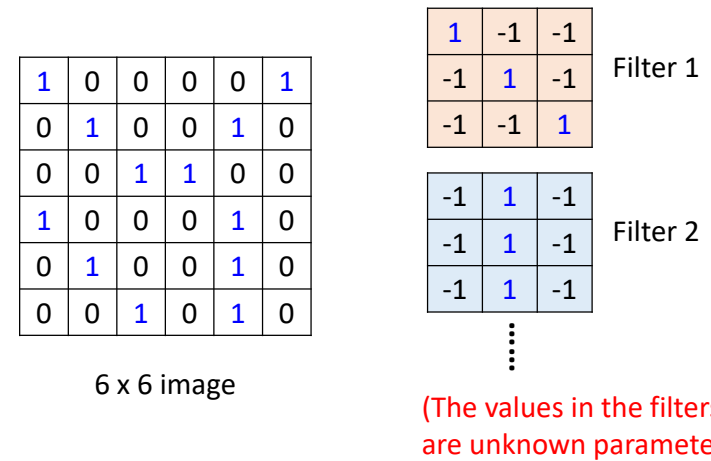
Convolutional Layer



9

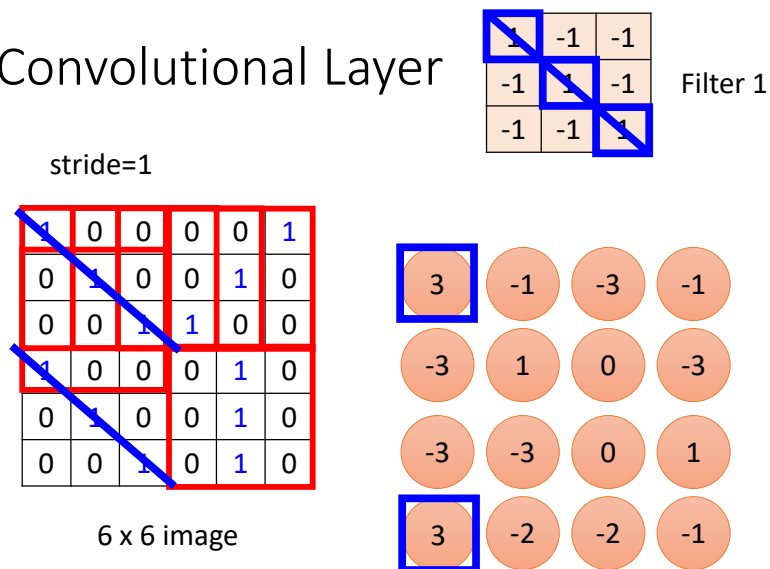
Convolutional Layer

Consider channel = 1
(black and white image)



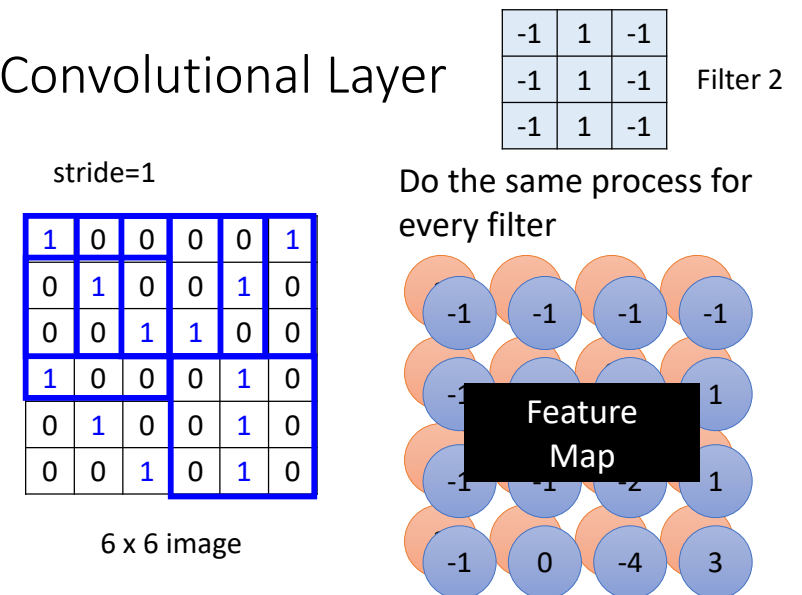
10

Convolutional Layer



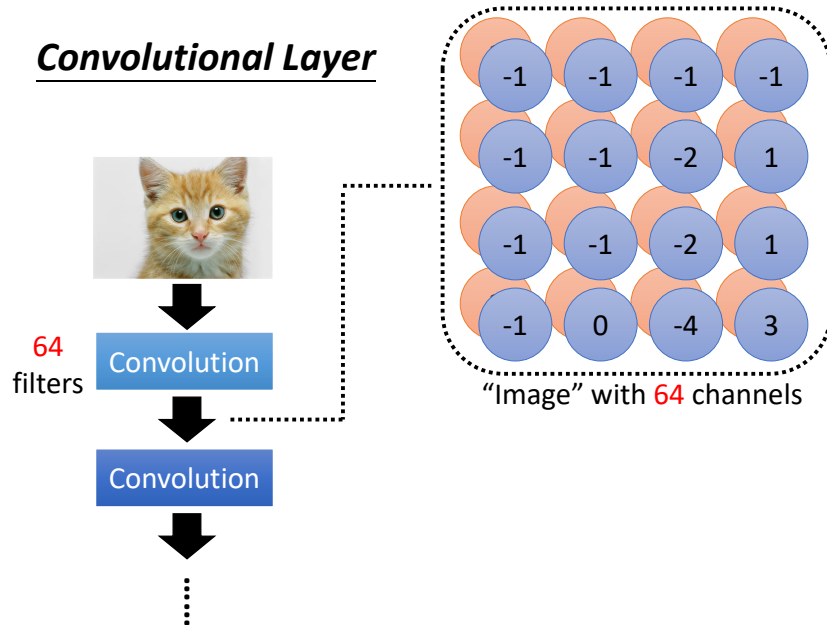
11

Convolutional Layer

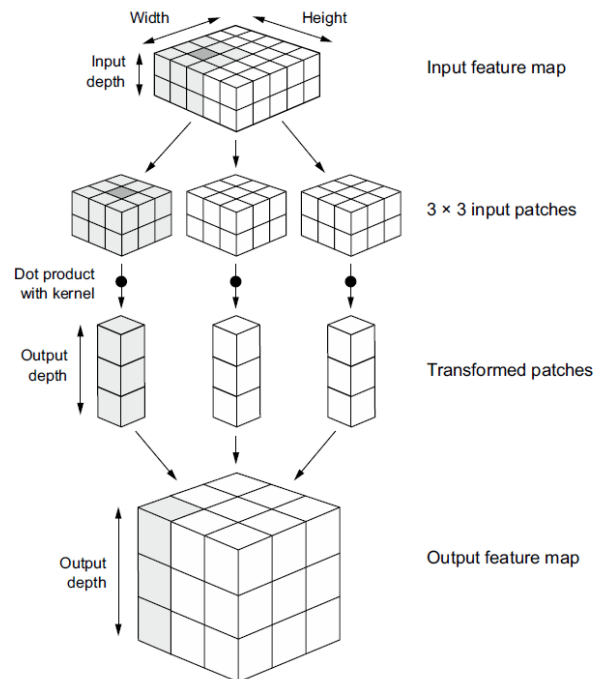
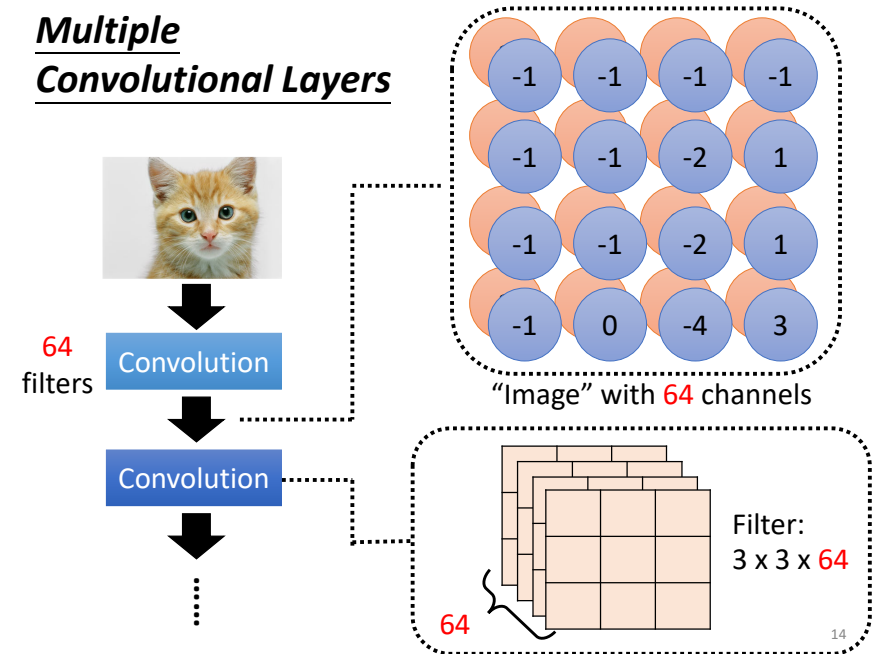


12

Convolutional Layer



Multiple Convolutional Layers



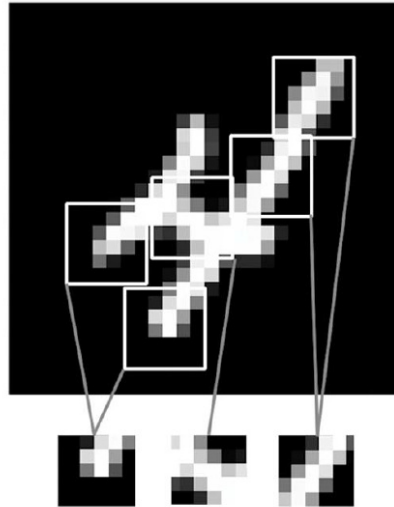
PURDUE
UNIVERSITY
FORT WAYNE

Why Convolution Operation?

- Two interesting properties
 - The patterns they learn are **translation-invariant**.
 - They can learn **spatial hierarchies** of patterns.

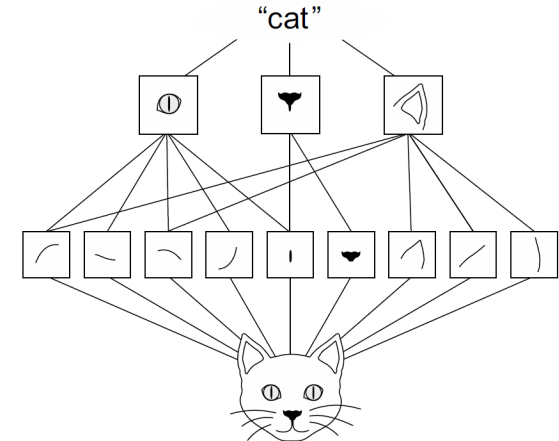
Translation-Invariant

- The *visual world is fundamentally translation-invariant*.
- Dense layers learn global patterns in their input feature space (for example, for a MNIST digit, patterns involving all pixels), whereas convolution layers learn **local patterns**—in the case of images, patterns found in small 2D windows of the inputs



Spatial Hierarchies of Patterns

- The *visual world is fundamentally spatially hierarchical*.

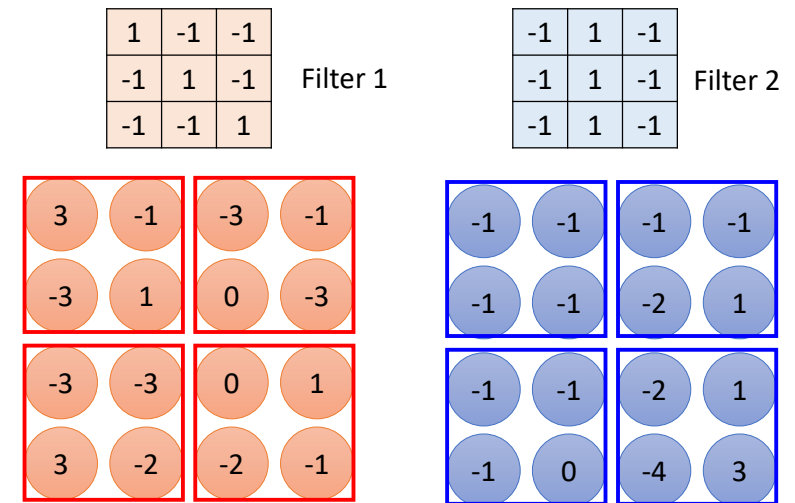


18

The Max-Polling Operation

- In Keras, **MaxPooling2D** layer.
- Aggressively **downsample** feature maps.
 - For example, the input feature map is 26×26 , the max-pooling operation halves it to 13×13 .

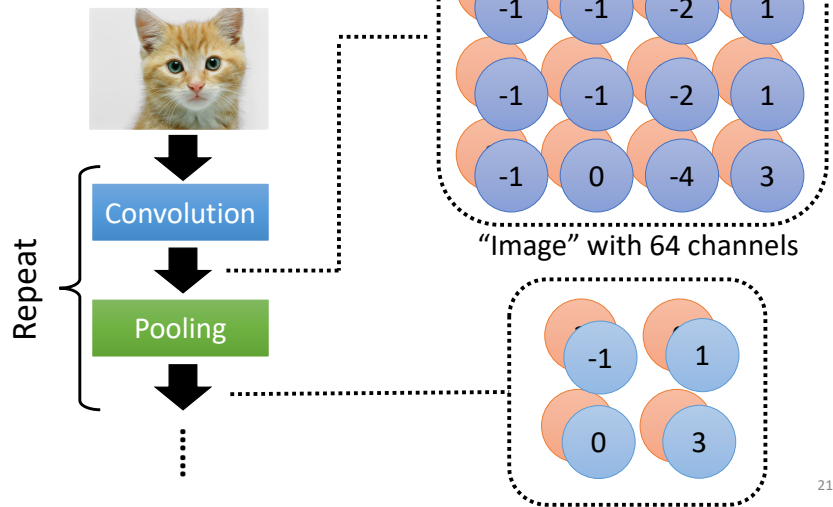
Pooling – Max Pooling



19

20

Convolutional Layers + Pooling



PURDUE UNIVERSITY FORT WAYNE Why Max-Pooling Layers?

- An example based on MNIST Digits
- With the max-pooling layers
 - **Reduce** the number of feature-map coefficients to process
 - **Induce** spatial-filter hierarchies by making successive convolution layers

22

PURDUE UNIVERSITY FORT WAYNE How Param # are Calculated?

Listing 8.2 Displaying the model's summary

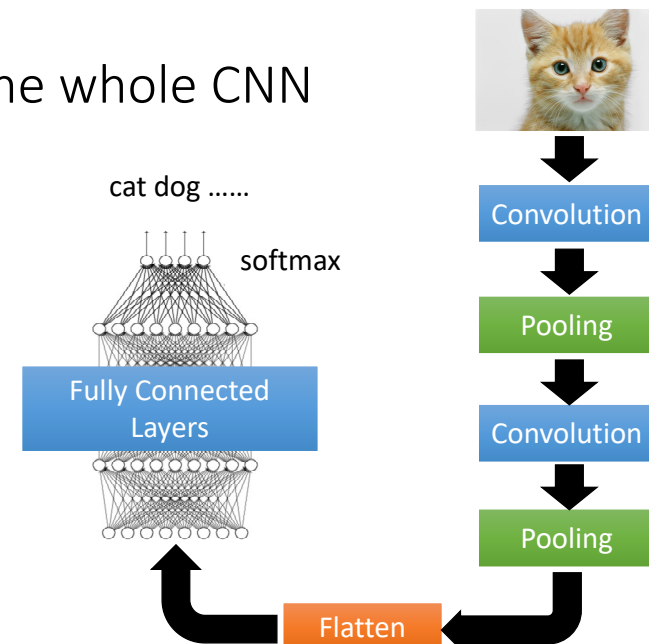
```
>>> model.summary()
Model: "model"
```

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 28, 28, 1)]	0
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_2 (Conv2D)	(None, 3, 3, 128)	73856
flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 10)	11530

Total params: 104,202
Trainable params: 104,202
Non-trainable params: 0

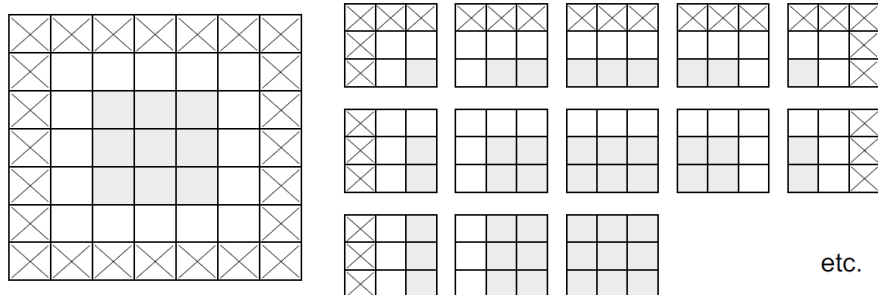
23

The whole CNN





Border Effects and Padding



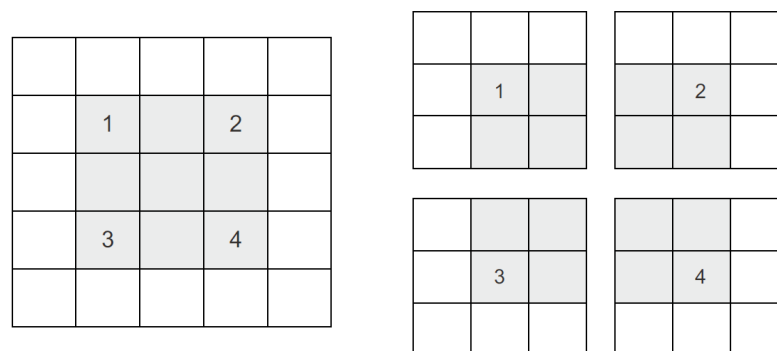
- In Conv2D layers, padding is configurable via the **padding** argument
 - “valid” (default): no padding
 - “same”: pad in such a way as to have an output with the same width and height as the input

25



Convolution Strides

- The distance between two successive windows is a parameter of the convolution, called its **stride**, which defaults to 1.



26



Outline

- Introduction to CNN
 - “Hello World” MNIST Digits Example
- **CNN for a Small Dataset**
 - **Classifying Images as Dogs or Cats Example**

27



Classifying Images as Dogs or Cats Example

- From dogs-versus-cats Kaggle competition
 - Only use 10% of original dataset
- Very little data
 - 2,000 pictures for training
 - 1,000 for validation
 - 2,000 for testing

28

Samples from Dataset



29

Methods

1. Train from scratch with no regularization
 - ~70% accuracy
2. Introduce data augmentation
 - ~80% - 85% accuracy
3. Feature extraction from pretrained model
 - ~97.5% accuracy
4. Fine-tune pretrained model
 - ~98.5% accuracy

30

Method 1: Train From Scratch with No Regularization

■ Steps

- Download the data
- Build the CNN model
- Preprocess the data
- Compile and train the model
- Evaluate the model

31

Method 2: Introduce Data Augmentation

- Data augmentation takes the approach of generating more training data from existing training samples by *augmenting* the samples via a number of **random transformations** that yield believable-looking images.
 - RandomFlip("horizontal")
 - Horizontal flip to a random 50% of the images
 - RandomRotation(0.1)
 - Rotate the input images by a random value in the range [-10%, +10%] (that is, [-36 degrees, +36 degrees])
 - RandomZoom(0.2)
 - Zoom in or out of the image by a random factor in the range [-20%, +20%]

32



33

Method 3: Feature Extraction from Pretrained Model

- What is a pretrained model?
 - A model that was previously trained on a **large dataset**, typically on a large-scale image-classification task.
- Apply DL for small-data problems
 - The **portability** of learned features across **different** problems is a key advantage of deep learning compared to many older, shallow learning approaches.
- Transfer learning

34

Well-known Pretrained Models

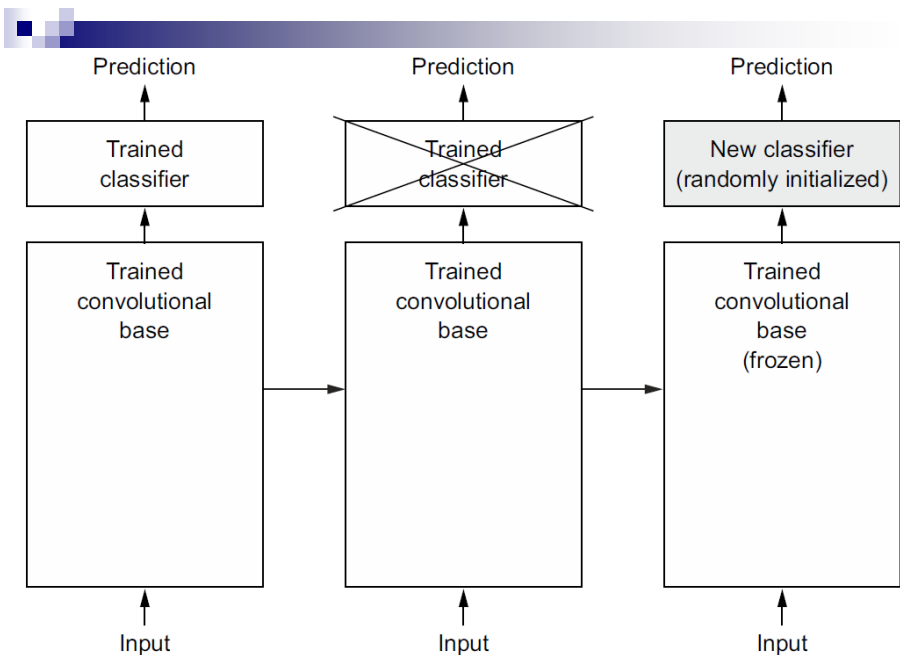
- Available as part of keras.applications:
 - VGG16
 - Xception
 - ResNet
 - MobileNet
 - EfficientNet
 - DenseNet
 - etc.

35

Two Ways to Use a Pretrained Model

- Feature extraction
 - A pretrained model for image classification comprises two parts:
 - **Convolutional base** and **densely connected classifier (top)**
 - Takes the convolutional base of a previously trained network, running the **new** data through it, and training a **new** classifier on top of the output.
 - Reason: the feature maps of a convnet are presence maps of **generic** concepts over a picture, which are likely to be useful regardless of the computer vision problem at hand.
- Feature-tuning

36



37

PURDUE UNIVERSITY FORT WAYNE Two Ways for Feature Extraction

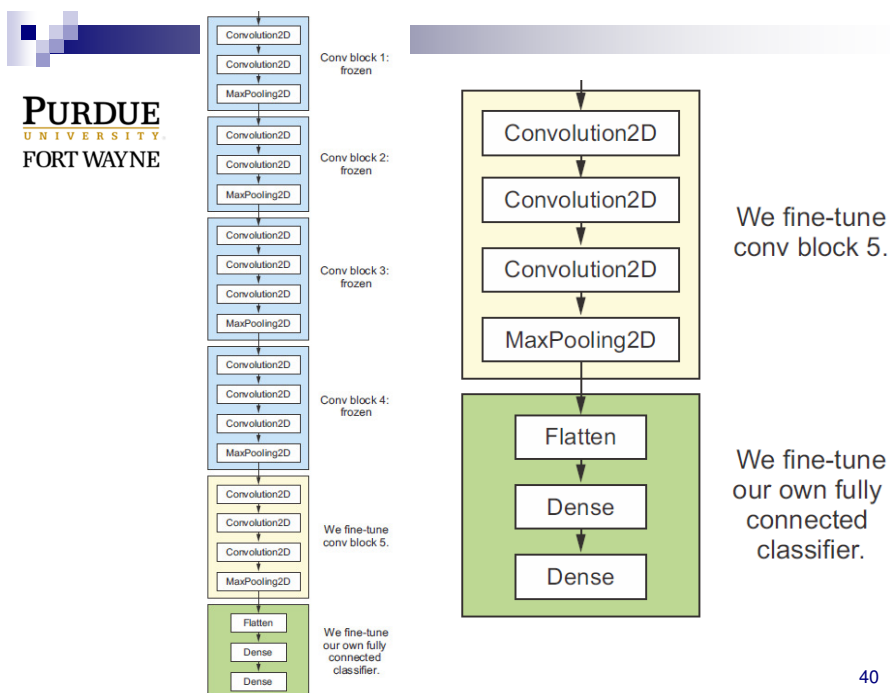
- Run the convolutional base over our dataset, **record** its **output** to a NumPy array on disk, and then use this data as input to a standalone, densely connected classifier.
 - Fast and cheap to run
- Extend the model we have (conv_base) by adding Dense layers on top, and run the whole thing from **end to end** on the input data.
 - Allow us to use data augmentation

38

PURDUE UNIVERSITY FORT WAYNE Method 4: Fine-tune Pretrained Model

- Idea
 - **Complementary** to feature extraction.
 - **Unfreezes** a few of the top layers of a frozen model base used for feature extraction.
 - **Jointly** trains both the newly added part of the model (in this case, the fully connected classifier) and these top layers.
- Only slightly **adjusts** the more abstract representations of the model being reused in order to make them more relevant for the problem at hand.

39



40



Steps

1. Add our custom network on top of an already-trained base network.
2. Freeze the base network.
3. Train the part we added.
4. **Unfreeze** some layers in the base network.
 - Note that should not unfreeze “batch normalization” layers
5. **Jointly train** both these layers and the part we added.

41



Why Not Fine-tune More Layers?

- You could!
- But consider
 - **Earlier layers** in the convolutional base encode more generic, reusable features, whereas layers higher up encode more specialized features.
 - The more parameters you’re training, the more you’re at risk of **overfitting**.
- It’s a **good** strategy to fine-tune only the top two or three layers in the convolutional base.

42



Summary

- Understand the basic operations in CNN
 - Convolution, max pooling, etc.
- Understand the CNN properties
 - Translation-invariant and spatial hierarchies of modular patterns
- Ways to apply CNN for a small dataset
 - Data augmentation
 - Pretrained models (i.e., transfer learning)
 - Feature extraction
 - Fine-tuning

43



Q & A

Have fun!!!